

SIMATS ENGINEERING ASSESSMENT TOOLS

COURSE NAME: Theory of Computation

COURSE CODE: CSA1304

COURSE OUTCOME COVERED

CO5: *Design Pushdown Automata and analyze their equivalence with Context-Free Grammars through formal construction and proof techniques.*

(BL : 3)

Assessment Tool Weightage: 10%

QUESTIONS: 20

Total Marks:20

Duration :1 Hour

Mock Interview question

Code of Conduct:

I SRIVARSHA M -192371010 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:SRIVARSHA M

SIMATS ENGINEERING ASSESSMENT TOOLS

Q1. What is a Pushdown Automaton (PDA)? How is it different from a Finite Automaton?

A Pushdown Automaton (PDA) is a computational model that extends a Finite Automaton (FA) by adding an auxiliary memory in the form of a stack. It is used to recognize Context-Free Languages (CFLs), which are a larger class of languages than the Regular Languages recognized by an FA.

The key difference lies in memory: an FA has no external memory and its next state depends only on the current state and input symbol, so it cannot "remember" arbitrarily long patterns. A PDA has a stack that can grow or shrink without bound, allowing it to keep track of unbounded information (such as counting nested symbols). This extra memory allows a PDA to accept languages like $a^n b^n$, which no FA can accept.

Q2. Define the components of a PDA formally.

A PDA is formally defined as a 7-tuple: $M = (Q, \Sigma, \Gamma, \delta, q_0, Z_0, F)$, where:

- Q : a finite set of states
- Σ : a finite set of input symbols (input alphabet)
- Γ : a finite set of stack symbols (stack alphabet)
- δ : the transition function, $\delta: Q \times (\Sigma \cup \{\epsilon\}) \times \Gamma \rightarrow \text{finite subsets of } Q \times \Gamma^*$
- q_0 : the initial state, $q_0 \in Q$
- Z_0 : the initial stack symbol, $Z_0 \in \Gamma$
- F : the set of final (accepting) states, $F \subseteq Q$

Q3. What is a stack in PDA? Why is it important?

The stack is a Last-In-First-Out (LIFO) auxiliary storage attached to the PDA. Symbols can be pushed onto the top of the stack or popped from it during a transition, and only the topmost symbol is accessible at any time.

The stack is important because it gives the PDA unbounded memory, which a plain finite automaton lacks. This allows the PDA to keep track of nested or balanced structures, such as matching parentheses or counting the number of a's to compare with the number of b's, which is essential for recognizing context-free languages.

Q4. What is an Instantaneous Description (ID) in PDA?

An Instantaneous Description (ID) represents the complete configuration of a PDA at a given moment during computation. It is written as a triple:

(q, w, γ)

where q is the current state, w is the remaining unread portion of the input string, and γ is the current content of the stack (usually written with the top symbol on the left). An ID captures everything needed to know how the PDA can proceed from that point onward.

Q5. What are the different types of moves in a PDA?

A PDA can make the following types of moves during a transition:

- Push move: a symbol is pushed onto the top of the stack, increasing its height.
- Pop move: the top symbol of the stack is removed, decreasing its height.
- No-change (replace) move: the top symbol is replaced by another symbol (or the same symbol), keeping the stack height unchanged.
- Epsilon (ϵ) move: the PDA changes state and/or manipulates the stack without reading any input symbol.

Q6. What is acceptance by final state and empty stack?

A PDA can accept an input string using one of two acceptance criteria:

- Acceptance by Final State: the string is accepted if, after consuming the entire input, the PDA is in one of the designated final states in F , regardless of the stack content.

SIMATS ENGINEERING ASSESSMENT TOOLS

- Acceptance by Empty Stack: the string is accepted if, after consuming the entire input, the stack becomes completely empty, regardless of the state the PDA is in.

Both methods are equally powerful — any language accepted by final state can be accepted by empty stack, and vice versa, using standard conversion constructions.

Q7. Define the language accepted by a PDA.

The language accepted by a PDA M by final state is defined as:

$$L(M) = \{ w \in \Sigma^* \mid (q_0, w, Z_0) \vdash^* (q, \varepsilon, \gamma) \text{ for some } q \in F \text{ and some stack string } \gamma \}$$

In words, $L(M)$ is the set of all input strings w that, starting from the initial ID, allow the PDA to reach some final state after consuming the entire string, through a sequence of valid moves.

Q8. How does a PDA process a string using IDs?

Processing a string is modeled as a sequence of moves between IDs. Starting from the initial ID (q_0, w, Z_0) , each move is written using the "turnstile" relation $\vdash$, where one ID transforms into the next by applying a transition rule of δ — consuming one input symbol (or none, for an ε -move) and updating the stack accordingly.

The relation $\vdash^*$ denotes zero or more such moves. The string w is accepted if there exists a sequence of moves $(q_0, w, Z_0) \vdash^* (q, \varepsilon, \gamma)$ that consumes all of w and ends in an accepting configuration, as defined by the acceptance condition (final state or empty stack).

Q9. Differentiate between Deterministic PDA and Non-Deterministic PDA.

A Deterministic PDA (DPDA) has, for every state, input symbol (or ε) and stack-top symbol, at most one applicable transition — there is no choice at any step, and if a move on ε is defined, no move on an input symbol is allowed from that same configuration.

A Non-Deterministic PDA (NPDA) allows multiple possible transitions from the same configuration, and the machine can explore several computation paths, accepting a string if at least one path leads to acceptance.

Unlike finite automata (where NFA and DFA are equally powerful), NPDAs are strictly more powerful than DPDAs: NPDAs accept all context-free languages, while DPDAs accept only a proper subset called Deterministic Context-Free Languages (DCFLs).

Q10. Why are PDAs equivalent to Context-Free Grammars (CFG)?

PDAs and CFGs are equivalent in expressive power because each can be systematically constructed from the other while preserving the language generated/accepted:

- CFG to PDA: any context-free grammar can be converted into an equivalent PDA that simulates leftmost derivations, using the stack to hold the portion of the sentential form yet to be matched.
- PDA to CFG: any PDA can be converted into an equivalent CFG by defining grammar variables that represent the possibility of moving between two stack configurations while reading a substring.

Because this conversion works in both directions, the class of languages generated by CFGs is exactly the class of languages accepted by PDAs — both define the Context-Free Languages.

Q11. What types of languages are accepted by PDA? Give examples.

PDAs accept exactly the class of Context-Free Languages (CFLs), which includes many languages that are not regular because they require counting or matching nested structures. Examples include:

- $L1 = \{ a^n b^n \mid n \geq 0 \}$ — equal numbers of a's followed by b's
- $L2 = \{ w w^R \mid w \in \{a, b\}^* \}$ — even-length palindromes
- $L3 = \text{balanced parentheses, e.g. } "(())"$
- $L4 = \{ a^i b^j c^k \mid i = j \text{ or } j = k \}$ — languages combining CFLs

SIMATS ENGINEERING ASSESSMENT TOOLS

Q12. What is a Turing Machine? How is it more powerful than PDA?

A Turing Machine (TM) is the most general computational model, consisting of a finite control, an infinite tape divided into cells, and a read/write head that can move left or right over the tape. It can read a symbol, write a new symbol in its place, change state, and move the head, on each step.

A TM is more powerful than a PDA because its tape acts as unrestricted, two-way, read-write memory, whereas a PDA's stack is a one-way, last-in-first-out memory that can only be accessed from the top. This lets a TM accept languages requiring more than nested counting, such as $\{ a^n b^n c^n \mid n \geq 0 \}$, which no PDA can accept. TMs define the class of Recursively Enumerable Languages, which strictly contains the Context-Free Languages accepted by PDAs.

Q13. Define the components of a Turing Machine.

A Turing Machine is formally defined as a 7-tuple: $M = (Q, \Sigma, \Gamma, \delta, q_0, B, F)$, where:

- Q : a finite set of states
- Σ : the input alphabet ($\Sigma \subseteq \Gamma$, and $B \notin \Sigma$)
- Γ : the tape alphabet, which includes Σ and the blank symbol
- δ : the transition function, $\delta: Q \times \Gamma \rightarrow Q \times \Gamma \times \{L, R\}$
- q_0 : the initial state, $q_0 \in Q$
- B : the blank symbol, $B \in \Gamma$
- F : the set of final (accepting) states, $F \subseteq Q$

Q14. Explain tape, head, and state transitions in a Turing Machine.

Tape: an infinite (or unbounded) sequence of cells, each capable of holding one symbol from the tape alphabet Γ . The tape serves as both the input and the working memory of the machine.

Head: a read/write head positioned over one cell of the tape at any time. On each step it reads the symbol under it, optionally writes a new symbol in its place, and then moves one cell Left (L) or Right (R).

State transitions: governed by the transition function $\delta(q, X) = (p, Y, D)$, meaning that in state q while reading symbol X , the machine moves to state p , writes Y in place of X , and moves the head in direction D . The computation proceeds step by step until the machine halts in a final state (accepting) or reaches a configuration with no defined transition (rejecting).

Q15. Explain programming techniques for Turing Machines.

Because a basic TM has only a single tape and simple move rules, several programming techniques are used to design TMs for complex tasks more conveniently:

- **Storage in finite control:** encoding a small, bounded amount of extra data as part of the state itself, so the machine can "remember" a symbol or flag without writing it to the tape.
- **Multiple tracks:** treating one physical tape as several logical tracks stacked at each cell, useful for marking positions or storing auxiliary information alongside the input.
- **Checking off symbols:** marking tape symbols as "used" (e.g., replacing or tagging them) to keep track of progress while comparing or matching symbols, common in verifying equal counts.
- **Shifting over:** making space on the tape by shifting a block of symbols left or right, useful for inserting new symbols.
- **Subroutines:** designing reusable blocks of states that perform a specific sub-task (like copying or comparing) and can be invoked from multiple places in the overall machine design.

Q16. What is storage in finite control in Turing Machines?

Storage in finite control is a design technique where a Turing Machine's state is used to hold a limited amount of additional information, effectively splitting each state into a pair (control state, stored symbol/data). This lets the machine "remember" something it has just read — such as a symbol it needs to compare later — without needing to write it onto the tape.

SIMATS ENGINEERING ASSESSMENT TOOLS

Since the number of states must remain finite, this technique can only store a bounded, fixed amount of information (for example, one symbol from a finite alphabet), not unbounded data; it is a convenience for machine design rather than a source of extra computational power.

Q17. Compare FA, PDA, and TM based on memory, power, and language acceptance.

Feature	Finite Automaton (FA)	Pushdown Automaton (PDA)	Turing Machine (TM)
Memory	No auxiliary memory	One stack (LIFO)	Infinite two-way tape
Computational Power	Weakest	Intermediate	Most powerful
Language Class Accepted	Regular languages	Context-free languages	Recursively enumerable languages
Corresponding Grammar	Regular grammar	Context-free grammar (CFG)	Unrestricted (Type-0) grammar
Example Language	Strings ending in 'ab'	$a^n b^n, n \geq 0$	$a^n b^n c^n, n \geq 0$

Q18. What is DFA?

A Deterministic Finite Automaton (DFA) is a finite automaton in which, for every state and every input symbol, there is exactly one defined transition to a next state — there is no ambiguity or choice at any step, and no ϵ -transitions are allowed. Formally, a DFA is a 5-tuple $(Q, \Sigma, \delta, q_0, F)$, where $\delta: Q \times \Sigma \rightarrow Q$ is a total function. A DFA accepts an input string if, starting from q_0 , the string drives the machine to a state in F after all symbols have been read. DFAs recognize exactly the class of Regular Languages.

Q19. Which language is accepted by a Turing Machine?

A Turing Machine accepts exactly the class of Recursively Enumerable (RE) languages, also called Turing-recognizable languages. A language L is recursively enumerable if there exists a TM that halts and accepts every string in L ; for strings not in L , the TM may either halt and reject or run forever without halting.

This class is the largest in the Chomsky hierarchy and strictly contains the Context-Free Languages (accepted by PDAs), the Regular Languages (accepted by FAs), and the smaller class of Recursive (decidable) languages, for which a TM is guaranteed to always halt.

Q20. How are Turing Machines applied in real-world applications?

Although Turing Machines are primarily a theoretical model, they underpin many practical areas of computer science:

- Theory of computability: TMs define what problems are algorithmically solvable, forming the basis of the Church–Turing thesis.
- Compiler and language design: understanding the limits of parsing and language recognition helps in designing compilers and interpreters.
- Algorithm analysis and complexity theory: the TM model underlies complexity classes such as P and NP, guiding the study of efficient versus intractable algorithms.
- Programming language semantics: TMs provide a formal basis for reasoning about what general-purpose programming languages can and cannot compute.
- Artificial intelligence and automata-based systems: concepts from TMs inform the design of parsers, interpreters, and formal verification tools used in compilers, security analysis, and protocol checking.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
----------	-----------	-------------------------------	--------------------------------	----------------------------------	--------------------------------

SIMATS ENGINEERING ASSESSMENT TOOLS

Conceptual Understanding	30	Demonstrates clear and complete understanding of the concept	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding
Accuracy of Answer	25	Completely correct answer with no errors	Mostly correct with minor errors	Partially correct with noticeable errors	Incorrect or irrelevant answer
Application of Knowledge	20	Correctly applies concepts to solve the question/problem	Applies concepts with minor mistakes	Limited or partially correct application	No or incorrect application
Completeness of Response	15	Covers all required parts of the question	Covers most parts with minor omissions	Covers only some parts	Incomplete or missing response
Clarity & Presentation	10	Clear, concise, and well-organized answer	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand